Aarhus university

Computer-Science

Distributed Systems and Security

Handin 2

Author	*Student number*
Søren M. Damsgaard	**202309814**
Alexander	**202408929**
Thor behrmann	**202410271**

February 28, 2026

Flooding and Ledgers

In this stage of evolution in our peer-to-peer network we have introduced Flooding and a Ledger system.
The design is mainly the same with the exception of the following:

- **Join Protocol:** This protocal is responsible for dynamically connecting all peers in the network when they join. Any joining peers sends a GetPeers message which create a two-way connection with the corresponding peer, which also sends a PeerList message back, which contains its address book for all peers it knows. The joining node then connects to any unknown nodes/address in the list, then floods the network with a Join message, updating all local address books.

- **Flooding:** To dissemniate transactions and any announcements we use dedouplicating for the flooding by assigning every message a global unqiue MsgID. All peers maintian a local seenMsgs map/list, keeping track of recieved messages and forwards them or ignoring any messages it has already seen, preventing infinite loops (broadcast storms)

- **Ledger:** The network now has a ledger containing the state of all accounts. Each peer has a local copy of this ledger which updates when a unique non-duplicate transaction is recieved. This version holds no bias towards overdrafts and executes any transactions delivered even if the accounts goes negative. Because the flooding mechanisme ensure every peer eventually receives every transactions once, the local ledger eventually converges to the same state, hence eventual consistencey. Note that this is not a perfect implementation which is explained later in the report.

How to run code

When code is unzipped, go into the terminal and make sure you are in the directory below

```
    .\Handin_2
```

and to run the code use the following commands in the terminal

```
    go build .
    go run .
```

this builds and runs all necessary files, including main.go which is supposed to be handin.go (according to the exercise), but we prefere to to keep it as is, if this is pure blasphemy, please comment.

Testing

besides reusing the automatic test from the previous handin, we haven't made any automatic testing yet for this version, but we have implemented some small checks in the main.go file by

comparing the maps P.LEDGER.ACCOUNT across the peers and also the size of the P.PEERS map to confirm eventual consistency in Ledgers and Flooding.
Automatic testing for this version is currently on the Backlog.

A lot of testing has been (proudly?) made while coding using print statements which has been commented out in this version.

Deduplicating Flooding

We actually partly achieve Eventual Consistency, but only if the network is fully formed before any transaction are made, if a new peer joins after the transactions are made, Eventual Consistency fails hard.

The reason it works is because we use the SEENMSGS map which is forwarded once to every neighbor. In a fully connected network this guarantees every peer will eventually recieve the transactions and update its ledger, which leads to identical ledgers.
But if a peer joins late, after transactions are completed, it will recieve the SEENMSGS map but it won't update its ledger, syncing it with the other peers ledger. The new Peers ledger will be empty until further transactions are made, which means eventual consistency will never be achieve after this point.

Negative Account

We don't believe it would the network would achieve eventual consistency if overdrafts are rejected.
In peer-to-peer networks messsages arrive at differetn times and often out of order. If we reject overdrafts the ledgers would ruin any chance of consistency.
Imagine this scenario: If we had a network consisting og Søren, Alex and Thor, where Søren had $100kr.$ in everyones ledgers. If Søren wants to send $100kr.$ to both Alex and Thor, then peer A and peer B creates a transactiosn each simultaneously. Depending on which peer first receives the transaction we get two scenario where, Thor has $100kr.$ and Alex has $0kr.$ OR Thor has $0kr.$ and Alex has $100kr.$ in all ledgers.
Because peers process messages in differetn order and reject overdrafts, the ledgers will never converge, breaking eventual consistency.

Backlog

- Automatic testing for transaction and eventual consistency
- Update ledger for new peers after transaction has been executed
- Pray TA won't get mad for not using his solution in previous feedback

- More colorful/fun report (memes?)
- Use of more technical terms.
- Fix documentation and useless comments

AI usage

We used AI to help make part of main.go in this assignment since nobody had a good idea on how to send so many transactions to the peers, specifically the sendTransactions(p *Peer) function
AI was also used in solving an issue with handling the unique IDs when sending the maps via getPeerList(addr string) in peer.go